SOFTWARE ENGINEERING · CHAPTER 09

Lab-09: Activity Diagrams

You model the workflows of the ITC Campus Library System from Labs 07 and 08 as UML activity diagrams (Mermaid flowcharts, exported as PNG) and implement the Java that mirrors them: borrowing, returning with fines, reservation fulfilment with concurrent notifications, a cancellable CSV import, and the membership-renewal business process with a unit test per path. The challenge adds timed overdue handling with a `@Scheduled` job, a state-vs-activity comparison and a diagram review.

UML 2.5.1 · Mermaid 11 JDK 25 · Maven 3.9 · JUnit 6 ≈ 6 hours 5 tasks + 1 challenge

How to work through this lab. Do the tasks in order; each builds on the previous. Read the *Goal*, follow the *Steps*, compare with the *Expected output*, and tick the *Acceptance checklist* (saved in your browser). Open a hint only when stuck for more than 10 minutes.

0 Setup — 20 min

Case study: the **ITC Campus Library System** (members borrow and return books, pay fines, reserve books that are on loan, receive notifications, and renew their membership yearly). You will reuse the class names from Lab-08 (`Member`, `Book`, `Copy`, `Loan`, `Reservation`, `Fine`, `Membership`).

1. Verify the tools. Diagrams are written as Mermaid text and exported to PNG with the Mermaid CLI (draw.io or PlantUML are accepted alternatives as long as you submit the source text too).

```
java -version        # 25
mvn -version         # 3.9.16
node -v              # 20+ (only needed for the Mermaid CLI)
npx -p @mermaid-js/mermaid-cli mmdc -i diagrams/test.mmd -o
diagrams/test.png
```

2. Create a Maven project `library-activities` with the layout below and this `pom.xml` core (JUnit 6 via the BOM, Surefire 3.5, preview features enabled for Task 3):

```
<properties>
  <maven.compiler.release>25</maven.compiler.release>
```

```
    <maven.compiler.enablePreview>true</maven.compiler.enablePreview>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencyManagement><dependencies>
    <dependency>
      <groupId>org.junit</groupId><artifactId>junit-bom</artifactId>
      <version>6.1.3</version><type>pom</type><scope>import</scope>
    </dependency>
  </dependencies></dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId><artifactId>junit-
jupiter</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build><plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId><version>3.5.4</version>
      <configuration><argLine>--enable-preview</argLine></configuration>
    </plugin>
  </plugins></build>
```

3. Folder layout:

```
library-activities/
├── README.md                  # guard tables, review notes, answers
├── diagrams/                  # one .mmd + one .png per task
│   ├── task1-borrow.mmd
│   ├── task2-return.mmd
│   ├── task3-reservation.mmd
│   ├── task4-import.mmd
│   ├── task5-renewal.mmd
│   └── challenge-overdue.mmd
└── src/
    ├── main/java/edu/itc/library/...
    └── test/java/edu/itc/library/...
```

4. Conventions used in every diagram of this lab (from the slides): `f-circ` initial node, `fr-circ` activity final, `cross-circ` flow final, `(["Verb noun"])` actions, `{" "}` decision and merge, `@{ shape: fork }` for fork and join, `subgraph` for swimlanes and regions, guards as `-->|"[condition]"|`.

1 Borrow a book: the basic flow

Easy — 30 min

GOAL

Draw the *Borrow a book* activity with one initial node, one activity final, verb-noun actions and exactly one decision, then export it as PNG.

STEPS

1. Create `diagrams/task1-borrow.mmd` starting from the starter text below. Use `flowchart TD`.
2. Add the actions, in this order: *Scan member card*, *Scan book copy*, *Check copy availability*, *Register loan*, *Print due-date slip*.
3. Add one decision after *Check copy availability* with the guards `[copy available]` and `[copy on loan]`. The second guard leads to the action *Offer reservation*.
4. Bring both paths together with a **merge** node before the activity final (do not let two edges enter the same action).
5. Export: `npx -p @mermaid-js/mermaid-cli mmdc -i diagrams/task1-borrow.mmd -o diagrams/task1-borrow.png`.
6. Write `edu.itc.library.LoanService.borrow(Member m, Copy c)` whose statements follow the same order as the actions (one comment per action name).

STARTER DIAGRAM TEXT

```
flowchart TD
  s@{ shape: f-circ, label: " " }
  a1(["Scan member card"])
  a2(["Scan book copy"])
  %% TODO: a3 Check copy availability, decision d, a4 Register loan,
  %% a5 Print due-date slip, a6 Offer reservation, merge m
  e@{ shape: fr-circ, label: " " }
  s --> a1 --> a2
  %% TODO: edges with guards [copy available] / [copy on loan]
```

```
package edu.itc.library;

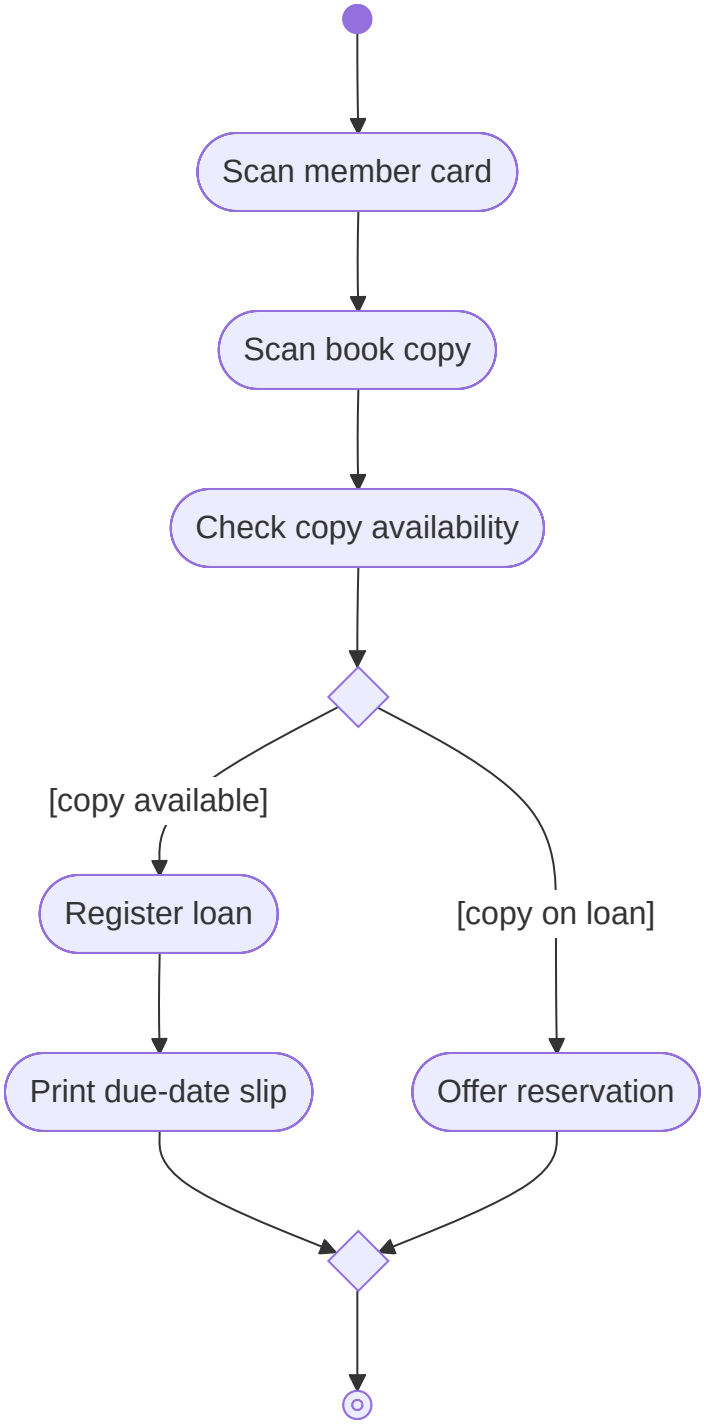
public class LoanService {
    private final LoanRepository loans;
    private final Clock clock;
    public LoanService(LoanRepository loans, Clock clock) {
        this.loans = loans; this.clock = clock;
    }

    public BorrowResult borrow(Member member, Copy copy) {
```

```
        // Scan member card / Scan book copy: already done by the caller
        // TODO Check copy availability (decision)
        // TODO [copy on loan] -> Offer reservation, return
BorrowResult.onLoan()
        // TODO [copy available] -> Register loan -> Print due-date slip
        return null;
    }
}
```

EXPECTED OUTPUT

Your rendered PNG should look like this (the layout may differ slightly):



Show hints

- A decision has one incoming edge and two or more outgoing guarded edges; a merge is the opposite. Both are `{" "}` in Mermaid.
- Guards go on the edges: `d -->|"[copy available]"| a4`. The quotes are needed because of the brackets.
- In `borrow()` the decision is an `if (copy.isAvailable())`; the merge is simply the code after the `if/else`.
- If `mmdc` complains about the `@{ shape: ... }` syntax, update the CLI: it needs Mermaid 11.3 or later.

ACCEPTANCE CHECKLIST

- [ ] `task1-borrow.mmd` and `.png` exist; exactly one initial and one activity final node.
- [ ] Every action is named verb + noun; the decision has two guards that are complete and exclusive.
- [ ] The alternative paths rejoin through a merge node, not through two edges into one action.
- [ ] `LoanService.borrow()` compiles and its statements follow the diagram's action order.

2 Return a book: decisions, guards and merges

Easy 45 min

GOAL

Model *Return a book* including the overdue-fine calculation with a three-way decision, write every guard as a Java boolean expression, and prove with unit tests that the guards are complete and exclusive.

STEPS

1. Create `diagrams/task2-return.mmd`: *Scan returned copy* → *Compute days overdue* → decision with three guards: `[days ≤ 0]`, `[0 < days ≤ 30]`, `[days > 30]`.
2. Paths: `[days ≤ 0]` → nothing; `[0 < days ≤ 30]` → *Charge daily fine* (0.25 per day); `[days > 30]` → *Charge lost-book fee* (book price + 7.50). All three paths rejoin at a merge.
3. After the merge: *Update copy status* → second decision `[reservation waiting]` / `[else]` → *Notify next member* → merge → activity final.
4. Create `FinePolicy` with `Money fineFor(LocalDate due, LocalDate returned, Money bookPrice)`. Write the three guards as private boolean methods (`onTime`, `overdueWithin30`, `lost`) and use them in the `if / else if / else`.

5. Write `FinePolicyTest` with one test per path plus the boundary days 0, 1, 30 and 31. Add a parameterised test that, for days −5..60, asserts **exactly one** guard method returns true.
6. Put a table "guard → Java expression → test name" into `README.md`; export the PNG.

STARTER CODE

```
package edu.itc.library;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

public class FinePolicy {
    static final BigDecimal DAILY = new BigDecimal("0.25");
    static final BigDecimal LOST_FEE = new BigDecimal("7.50");

    public BigDecimal fineFor(LocalDate due, LocalDate returned,
                              BigDecimal bookPrice) {
        long days = ChronoUnit.DAYS.between(due, returned); // Compute days
overdue
        if (onTime(days)) {                 // [days <= 0]
            return BigDecimal.ZERO;
        } else if (overdueWithin30(days)) { // [0 < days <= 30]
            return null;                    // TODO Charge daily fine
        } else {                            // [days > 30]
            return null;                    // TODO Charge lost-book fee
        }
    }
    boolean onTime(long days)          { return days <= 0; }
    boolean overdueWithin30(long days) { return false; }   // TODO
    boolean lost(long days)            { return false; }   // TODO
}


@ParameterizedTest
@ValueSource(longs = {-5, 0, 1, 15, 30, 31, 60})
void exactlyOneGuardIsTrue(long days) {
    var p = new FinePolicy();
    int trueGuards = (p.onTime(days) ? 1 : 0)
                   + (p.overdueWithin30(days) ? 1 : 0)
                   + (p.lost(days) ? 1 : 0);
    assertEquals(1, trueGuards, "guards must be complete and exclusive");
}
```

EXPECTED OUTPUT

```
$ mvn -q test
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0

README.md (excerpt)
| Guard               | Java expression          | Test
|
|---------------------|--------------------------|------------------------------
|
| [days ≤ 0]          | days <= 0                | returnedOnTimeHasNoFine
|
| [0 < days ≤ 30]     | days > 0 && days <= 30    | overdue30DaysCosts7_50
|
| [days > 30]         | days > 30                | overdue31DaysIsLostBook
|
```

Show hints

- Guards must not overlap: `[days > 0]` and `[days > 30]` would both be true for day 45. Write the upper bound explicitly.
- Mermaid edge labels: `-->|"[0 < days ≤ 30]"|`; use `#lt;` instead of `<` if your renderer eats the character.
- `BigDecimal.valueOf(days).multiply(DAILY)` for the daily fine; `bookPrice.add(LOST_FEE)` for the lost book.
- The second decision (`[reservation waiting]`) is a good place to reuse a merge before the final node; two edges into ⊙ are allowed, but be consistent.

ACCEPTANCE CHECKLIST

- [ ] The diagram has a three-way decision whose guards are complete and exclusive, and every alternative path rejoins through a merge.
- [ ] Each guard exists as a boolean method in `FinePolicy` and the `if / else if / else` uses them in the diagram's order.
- [ ] Tests cover every path and the boundaries 0, 1, 30, 31; the "exactly one guard" parameterised test passes.
- [ ] `README.md` contains the guard → expression → test table; PNG exported.

3 Reservation fulfilment: fork, join and swimlanes **Medium**

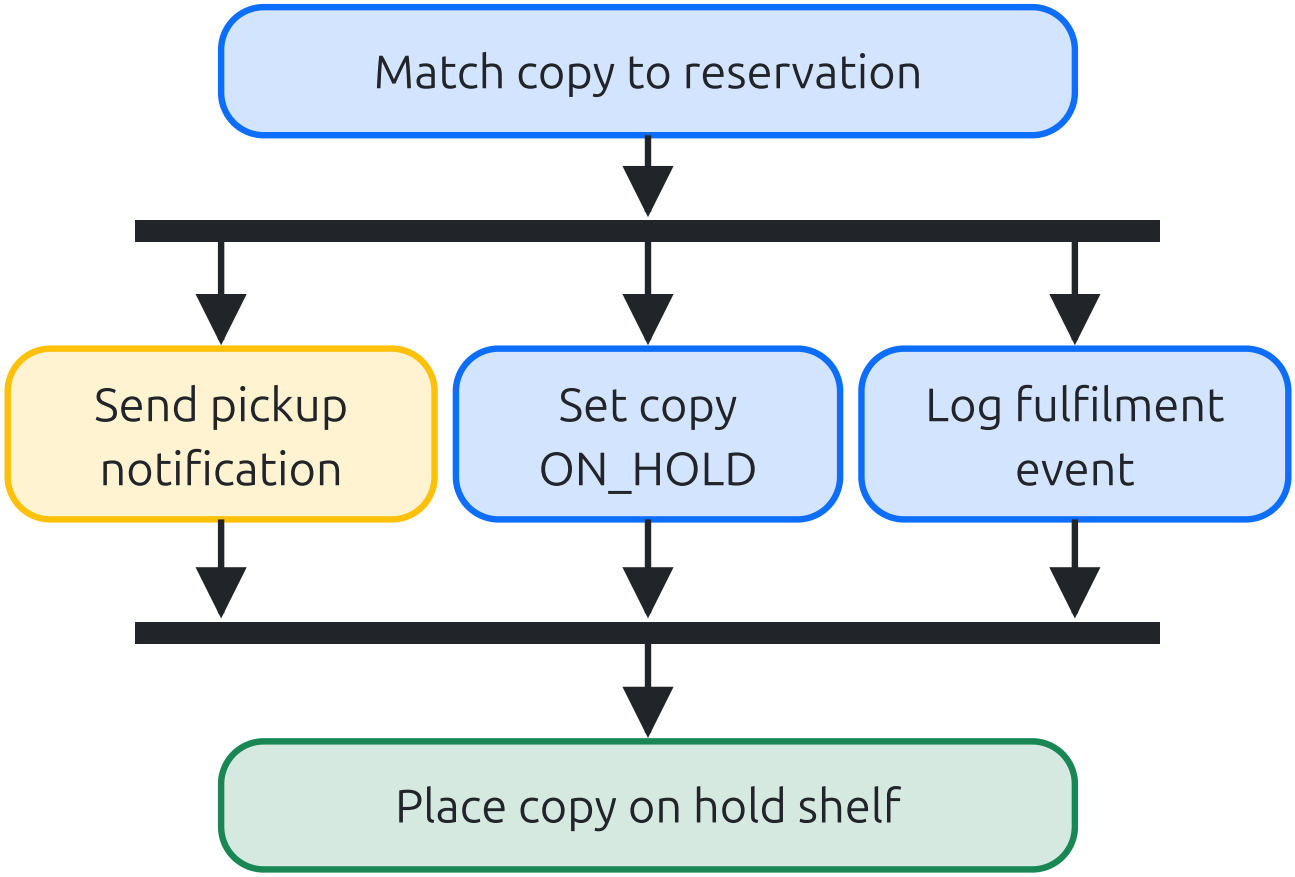
60 min

GOAL

When a reserved book is returned, three things happen at once: the member is notified, the catalogue is updated and the event is logged. Model this with a fork/join across four swimlanes and implement it with structured concurrency.

STEPS

1. Create `diagrams/task3-reservation.mmd` with `flowchart TB` and four `subgraph` lanes: **Member**, **Librarian**, **System**, **Notification service**.
2. System lane: *Match returned copy to oldest reservation* → fork. Branches: *Send pickup notification* (Notification service) || *Set copy status ON_HOLD* (System) || *Log fulfilment event* (System). Join, then Librarian: *Place copy on hold shelf*.
3. Member lane: *Collect book within 3 days* → decision `[collected]` → *Register loan* (Librarian) → final; `[not collected]` → System: *Release hold* → back to the merge before *Match …* (loop to the next reservation).
4. Implement `ReservationService.fulfil(Copy copy)` with `StructuredTaskScope.open()` (JDK 25 preview; the pom already enables it): three `fork` calls, one `join()`, then `holdShelf.place(copy)`. Also provide `fulfilWithFutures` using `CompletableFuture.allOf` and compare in `README.md`.
5. Test with fake collaborators that record the calling thread name; assert the three branches ran on different threads and that `place()` was called only after all three completed (use a `CountDownLatch` or timestamps).
6. Export the PNG; check that every fork has a join and that the loop uses a merge node.



join waits for all three tokens

STARTER CODE

```
flowchart TB
  subgraph M[Member]
    a7(["Collect book within 3 days"])
  end
  subgraph L[Librarian]
    a6(["Place copy on hold shelf"])
    a8(["Register loan"])
  end
  subgraph S[System]
    s@{ shape: f-circ, label: " " }
    m0{" "}
    a1(["Match returned copy to oldest reservation"])
    f@{ shape: fork, label: " " }
    %% TODO a3 Set copy status ON_HOLD, a4 Log fulfilment event,
    %% j join, d decision, a9 Release hold, e final
  end
  subgraph N[Notification service]
    a2(["Send pickup notification"])
  end
  s --> m0 --> a1 --> f
```

```
    f --> a2
    %% TODO remaining edges; [not collected] must go back to m0
```

```
package edu.itc.library;

import java.util.concurrent.StructuredTaskScope;

public class ReservationService {
    private final ReservationRepository reservations;
    private final Notifier notifier;
    private final Catalogue catalogue;
    private final EventLog log;
    private final HoldShelf holdShelf;
    // constructor omitted

    public void fulfil(Copy copy) throws InterruptedException {
        Reservation r = reservations.oldestFor(copy.book())
                .orElseThrow();                          // Match copy
        try (var scope = StructuredTaskScope.open()) {  // fork
            scope.fork(() -> { notifier.pickupReady(r); return null; });
            scope.fork(() -> { catalogue.setStatus(copy, Status.ON_HOLD);
                               return null; });
            scope.fork(() -> { log.fulfilled(r, copy); return null; });
            scope.join();                                   // join
        }
        holdShelf.place(copy, r);                       // Place on shelf
    }
}
```

EXPECTED OUTPUT

```
$ mvn -q test
notify   on thread ForkJoinPool-1-worker-1   t=0.002s
catalog  on thread ForkJoinPool-1-worker-2   t=0.002s
log      on thread ForkJoinPool-1-worker-3   t=0.002s
shelf    on thread main                      t=0.104s   (after all three)
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
```

Show hints

- Subtasks of `StructuredTaskScope` run on virtual threads; the thread names in your log will differ from the sample, only the distinctness matters.
- `scope.join()` throws `StructuredTaskScope.FailedException` if any branch throws; test that the shelf is *not* updated in that case.

- Only put truly independent work in the fork: *Place copy on hold shelf* depends on the status update, so it comes after the join.
- The loop back to `m0` is the merge that also receives the initial token; without it, the diagram has an implicit join at *Match* ….
- For the `CompletableFuture` variant, pass an explicit executor and shut it down in an `@AfterEach`.

ACCEPTANCE CHECKLIST

- [ ] Four swimlanes; each action sits in the lane of the party that performs it.
- [ ] One fork with three branches and one matching join; the "not collected" loop goes through a merge node.
- [ ] `fulfil()` uses `StructuredTaskScope` (and `fulfilWithFutures()` uses `CompletableFuture.allOf`); both pass the ordering test.
- [ ] The failure test proves the shelf is not updated when one branch throws; `README.md` compares the two implementations.

4 Bulk import: object flows, loops and exceptions **Medium** **60 min**

GOAL

The librarian imports a CSV file of new books. Model the import with object nodes (`File → Record → Book`), an expansion region per row, an interruptible region for "cancel import" and an exception handler for invalid rows; then implement the same loop in Java with `try/catch` and an import summary.

STEPS

1. Create `diagrams/task4-import.mmd`. Object nodes are plain rectangles (`o1["File"]`, `o2["Record"]`, `o3["Book"]`) styled with a `classDef obj`; actions stay rounded.
2. Flow: *Open CSV file* → `File` → expansion region `«iterative» for each row` containing *Parse row* → `Record` → *Validate record* → *Create book* → `Book` → *Save book*. After the region: *Write import summary* → activity final.
3. Draw the expansion region inside a dashed *interruptible region* that also contains the accept event *Cancel import*. Its lightning edge (`-. "⚡" .->`) leads to *Roll back current batch* → flow final.
4. Add an exception handler on *Validate record*: `InvalidRecordException` → *Record error and skip row* whose output joins the loop again (dashed edge labelled with the exception type).

5. Implement `BookImporter.importFrom(Path csv, BooleanSupplier cancelled)` returning `ImportSummary(int imported, int skipped, List<String> errors)`. Check `cancelled.getAsBoolean()` at the top of every iteration; catch `InvalidRecordException` per row.
6. Test with `src/test/resources/books.csv` (10 rows, 2 invalid: missing ISBN, negative price) and with a `cancelled` supplier that flips to true after 4 rows. Export the PNG.

STARTER CODE

```
package edu.itc.library.imports;

import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.function.BooleanSupplier;

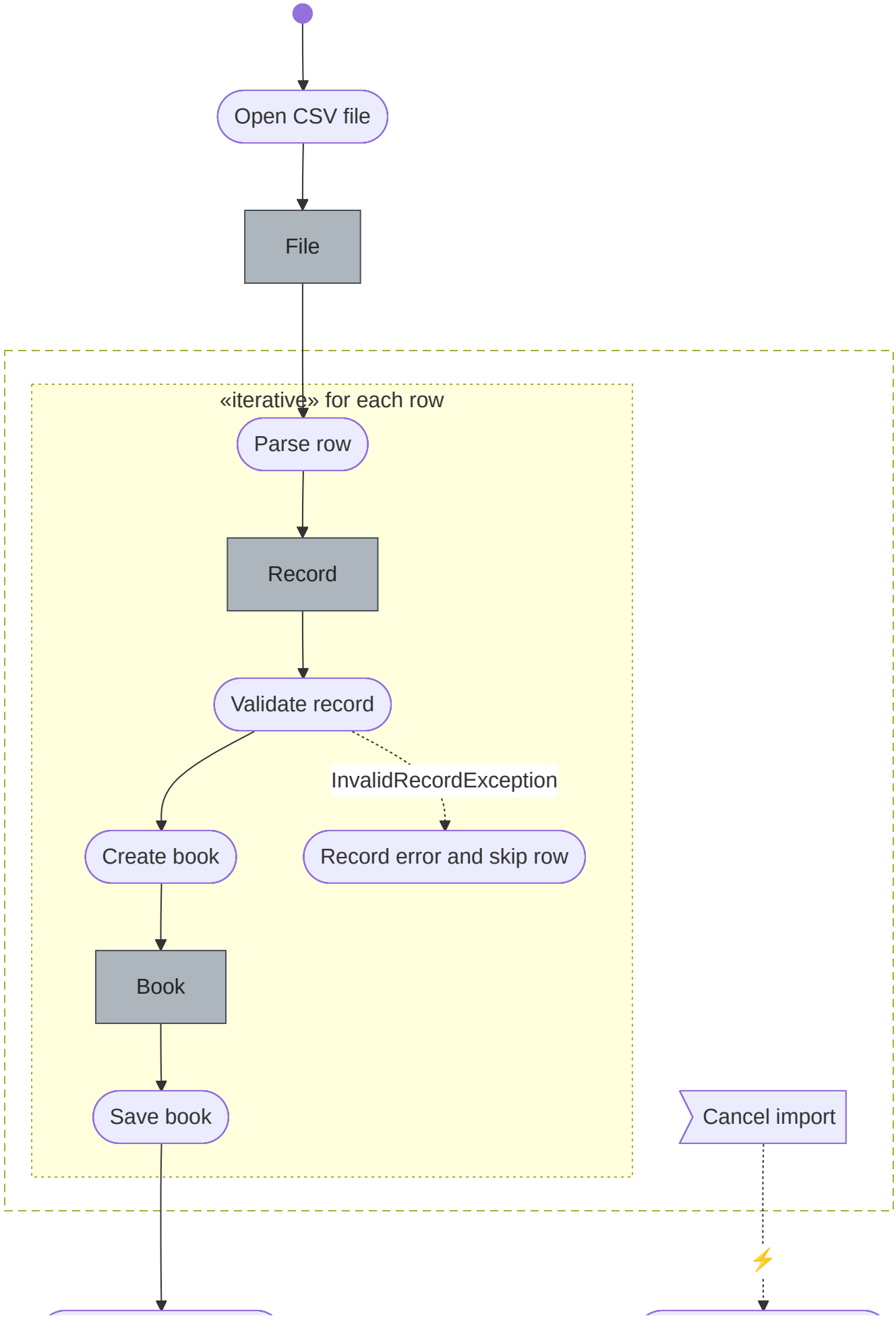
public class BookImporter {
    private final BookRepository books;
    private final RecordParser parser;
    public BookImporter(BookRepository books, RecordParser parser) {
        this.books = books; this.parser = parser;
    }

    public ImportSummary importFrom(Path csv, BooleanSupplier cancelled)
            throws java.io.IOException {
        int imported = 0, skipped = 0;
        List<String> errors = new ArrayList<>();
        List<String> lines = Files.readAllLines(csv);      // Open CSV file ->
File
        for (int i = 1; i < lines.size(); i++) {           // «iterative» for
each row
            if (cancelled.getAsBoolean()) {                 // accept event:
Cancel import
                // TODO Roll back current batch; stop the loop (flow final)
                break;
            }
            try {
                Record rec = parser.parse(lines.get(i));   // Parse row ->
Record
                // TODO Validate record (throws InvalidRecordException)
                // TODO Create book -> Book, Save book, imported++
            } catch (InvalidRecordException e) {           // exception handler
                // TODO Record error and skip row
            }
        }
        return new ImportSummary(imported, skipped, errors); // Write import
```

```
    summary
        }
    }

    public record ImportSummary(int imported, int skipped, List<String> errors) {}
```

EXPECTED OUTPUT


Open CSV file
File
«iterative» for each row
Parse row
Record
Validate record
InvalidRecordException
Create book
Record error and skip row
Book
Save book
Cancel import
⚡




```
$ mvn -q test
ImportSummary[imported=8, skipped=2, errors=[row 4: missing ISBN, row 9:
negative price]]
ImportSummary[imported=4, skipped=0, errors=[]]   (cancelled after 4 rows)
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
```

Show hints

- Nested `subgraph`s work in Mermaid: the interruptible region is the outer one, the expansion region the inner one.
- The exception handler edge is dashed (`-. label .->`) and labelled with the exception type; its target is inside the region because the loop continues with the next row.
- A `BooleanSupplier` lets the test flip cancellation deterministically: `int[] n = {0}; BooleanSupplier c = () -> ++n[0] > 4;`.
- Do not catch `Exception`; catching only `InvalidRecordException` is what the handler's type label promises.
- Skip the header line (`i = 1`) and report row numbers as they appear in the file so the librarian can find them.

ACCEPTANCE CHECKLIST

- [ ] Object nodes `File`, `Record`, `Book` are drawn as rectangles on object flows between the right actions.
- [ ] The «iterative» expansion region sits inside an interruptible region with an accept event and a lightning edge to a flow final.
- [ ] An exception handler for `InvalidRecordException` is drawn and implemented as a per-row `catch`; the summary counts imported, skipped and errors.
- [ ] Tests cover the 8/2 case and the cancellation after 4 rows; PNG exported.

5 Membership renewal: from business process to code Hard

90 min

GOAL

Model the *Renew membership* business process end-to-end (Member, Front desk, System, Payment provider), then implement `RenewalService.renew()` so that its method structure mirrors the diagram, with one unit test per path.

STEPS

1. Create `diagrams/task5-renewal.mmd` with lanes **Member**, **Front desk**, **System**, **Payment provider**. Member: *Request renewal*. System: *Load membership* → decision 1: `[account suspended]` → Front desk: *Refuse renewal* → activity final (rejected); `[else]` continues.
2. Decision 2 (System): `[outstanding fines > 0]` → Member: *Pay outstanding fines* → merge; `[else]` → merge.
3. Decision 3 (System) on membership type with three guards `[STUDENT]`, `[STAFF]`, `[EXTERNAL]` → *Apply student fee (5.00)* / *Apply staff fee (0.00)* / *Apply external fee (25.00)* → merge → Payment provider: *Collect payment* (skip via guard `[fee = 0]`).
4. Fork (System): *Extend expiry by 12 months* || *Issue receipt* || *Send confirmation email* → join → activity final (renewed).
5. Implement `RenewalService.renew(RenewalRequest req)` returning a sealed `RenewalResult` (`Renewed`, `Refused`). Each decision is an `if` or `switch`, each fork a `StructuredTaskScope` (or `CompletableFuture.allOf`), each merge is the code after the block. Put a comment with the action name on every step.
6. Write `RenewalServiceTest` with one test per path: suspended, fines then student, staff (no payment), external with card, external whose payment is declined (exception handler → `Refused`). Count the paths in your diagram and make the test count match. Export the PNG.

STARTER CODE

```
package edu.itc.library.membership;

import java.math.BigDecimal;
import java.time.LocalDate;

public sealed interface RenewalResult permits Renewed, Refused {}
public record Renewed(LocalDate newExpiry, BigDecimal feePaid)
        implements RenewalResult {}
public record Refused(String reason) implements RenewalResult {}

public class RenewalService {
    private final MembershipRepository memberships;
    private final FineService fines;
```

```
    private final PaymentProvider payments;
    private final ReceiptService receipts;
    private final Mailer mailer;
    // constructor omitted

    public RenewalResult renew(RenewalRequest req) throws InterruptedException {
        Membership m = memberships.load(req.memberId());   // Load membership
        if (m.isSuspended()) {                             // [account
suspended]
            return new Refused("account suspended");       // Refuse renewal
        }
        if (fines.outstanding(m).signum() > 0) {           // [outstanding fines
> 0]
            // TODO Pay outstanding fines (may throw PaymentDeclinedException)
        }
        BigDecimal fee = switch (m.type()) {               // decision 3
            case STUDENT  -> new BigDecimal("5.00");        // Apply student fee
            case STAFF    -> BigDecimal.ZERO;               // Apply staff fee
            case EXTERNAL -> new BigDecimal("25.00");       // Apply external
fee
        };
        // TODO [fee > 0] Collect payment; handler: PaymentDeclinedException ->
Refused
        // TODO fork: Extend expiry || Issue receipt || Send confirmation email;
join
        return null;
    }
}


class RenewalServiceTest {
    // fakes: InMemoryMemberships, FakeFines, FakePayments (can be told to
decline)

    @Test void suspendedAccountIsRefused() { /* path 1 */ }
    @Test void finesArePaidBeforeStudentRenewal() { /* path 2 */ }
    @Test void staffRenewsWithoutPayment() { /* path 3 */ }
    @Test void externalMemberPaysByCard() { /* path 4 */ }
    @Test void declinedPaymentRefusesRenewal() { /* path 5 */ }
}
```

EXPECTED OUTPUT

```
$ mvn -q test
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
```

```
diagrams/task5-renewal.png: 4 lanes, 3 decisions, 1 fork + 1 join,
2 activity finals (rejected, renewed), 12 actions, one initial node.
```

Show hints

- Draw first, code second. If a step in the code has no action in the diagram (or vice versa), one of them is wrong.
- Two activity finals are fine: label them with a short Mermaid node id like `eRej` and `eOk` and mention "rejected" / "renewed" in a note.
- Exception handler: wrap only *Collect payment* in the `try`; the `catch (PaymentDeclinedException e)` returns `new Refused("payment declined")`.
- The switch over the enum is exhaustive, which is exactly the "guards complete" rule; a fourth membership type would fail to compile until you add its guard.
- Keep the fork branches side-effect independent: none of them should read what another writes.

ACCEPTANCE CHECKLIST

- [ ] The diagram covers the whole process with four lanes, three decisions with complete and exclusive guards, one fork/join and two activity finals.
- [ ] `renew()` reads top-to-bottom like the diagram: same action names in comments, same order, decisions as `if`/`switch`, fork as structured concurrency.
- [ ] The declined-payment path is drawn as an exception handler and implemented as a `catch` that returns `Refused`.
- [ ] Exactly one test per path (5); all green from a clean checkout.

★ Challenge: overdue handling with timed events Hard

90–120 min · optional, extra credit

SCENARIO

The library wants an automated overdue workflow: 3 days after the due date a reminder is sent, from day 7 a daily fine accrues, and on day 30 the membership is suspended and a lost-book invoice is issued. The workflow ends when the book is returned at any point. The head librarian also hands you a cluttered diagram from a previous intern to review.

REQUIREMENTS

1. **Diagram** `diagrams/challenge-overdue.mmd`: use accept time events (`@{ shape: hourglass }`) for "due date + 3 days", "+ 7 days" and "+ 30 days", one interruptible region

around the whole escalation with the accept event *Book returned* whose lightning edge leads to *Close overdue case* → activity final. Lanes: System, Notification service, Member.

2. **Implementation sketch** `OverdueJob` with `@Scheduled(cron = "0 0 6 * * *")` that loads all open loans, computes `daysOverdue` and dispatches with a `switch` using guards `[3 ≤ d < 7]`, `[7 ≤ d < 30]`, `[d ≥ 30]` (idempotent: sending the same reminder twice must be impossible; store what was already sent on the `Loan`).
3. **State vs activity**: draw the `Loan` lifecycle as a Mermaid `stateDiagram-v2` (ACTIVE → OVERDUE → REMINDED → FINED → SUSPENDED, all → RETURNED) and write half a page in `README.md` on what each diagram shows that the other cannot (states and events vs actions and flows; which one you would give to the developer, which one to the librarian).
4. **Review report**: the intern's diagram is described below. List every problem you find against the readability checklist from the slides (one line each, with the rule violated), then redraw it as `diagrams/challenge-review.mmd`.

The intern's diagram (as described): 23 actions in one diagram; two initial nodes (one for "daily job", one for "manual check"); actions named *Processing*, *Check*, *Stuff for fines*, *Email*; a decision with guards *[ok]* and *[maybe]* and no *[else]*; a fork with four branches of which only two reach a join; the "Book returned" case drawn as a normal edge out of every action (14 edges to the same node); a lane called *Misc*; no final node; edges crossing each other 9 times.

SKELETON

```
package edu.itc.library.overdue;

import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

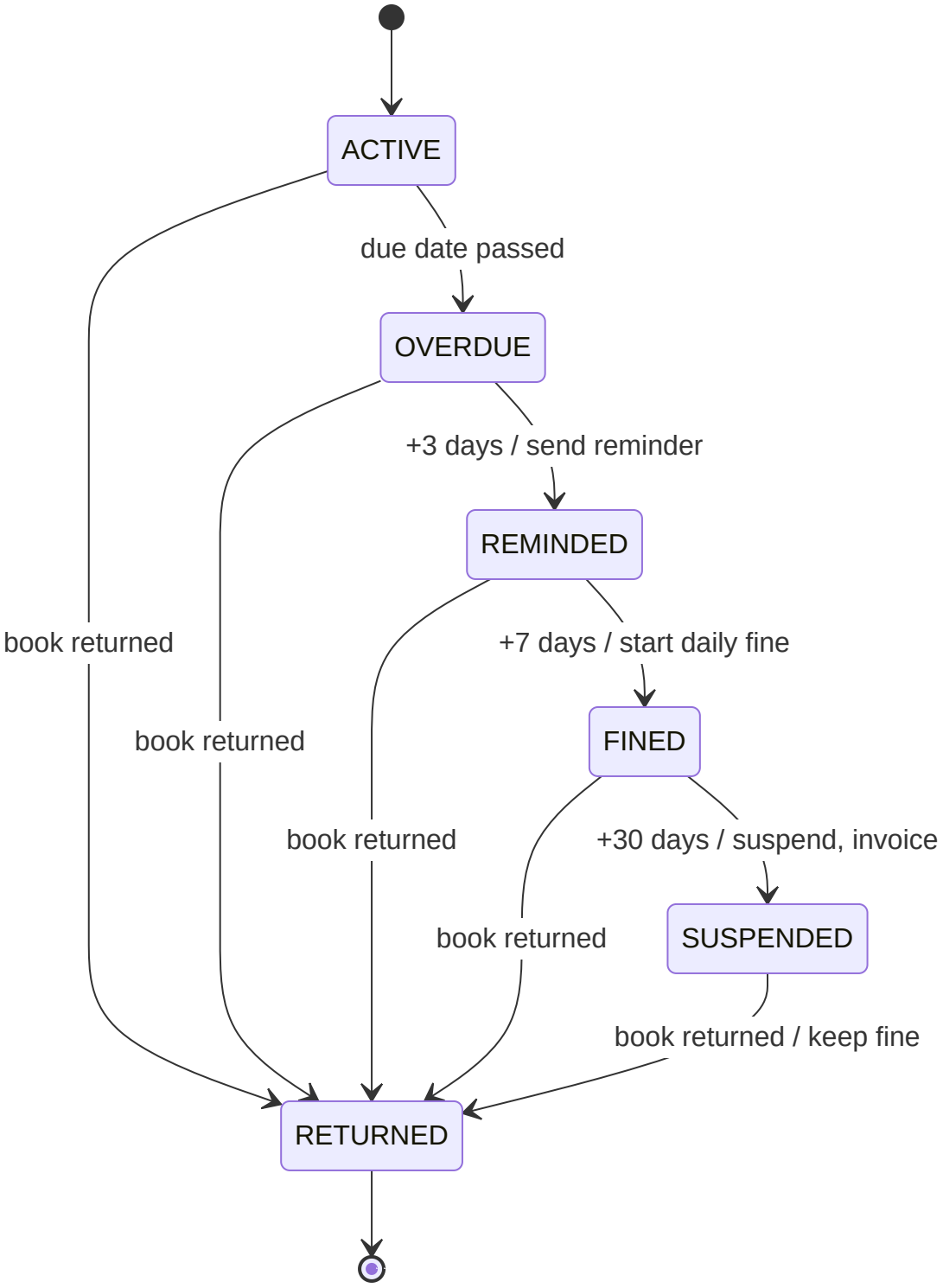
@Component
public class OverdueJob {
    private final LoanRepository loans;
    private final Notifier notifier;
    private final FineService fines;
    private final MembershipService memberships;
    // constructor omitted

    // accept time event: every day at 06:00
    @Scheduled(cron = "0 0 6 * * *", zone = "Asia/Phnom_Penh")
    public void run() {
        for (Loan loan : loans.findOpenOverdue()) {          // «iterative»
            long d = loan.daysOverdue(java.time.LocalDate.now());
            // TODO guards [3 <= d < 7] reminder, [7 <= d < 30] fine,
            //      [d >= 30] suspend + invoice; each step idempotent
        }
```

```
    }
}
```

```
stateDiagram-v2
  [*] --> ACTIVE
  ACTIVE --> OVERDUE : due date passed
  OVERDUE --> REMINDED : +3 days / send reminder
  REMINDED --> FINED : +7 days / start daily fine
  FINED --> SUSPENDED : +30 days / suspend, invoice
  ACTIVE --> RETURNED : book returned
  OVERDUE --> RETURNED : book returned
  REMINDED --> RETURNED : book returned
  FINED --> RETURNED : book returned
  SUSPENDED --> RETURNED : book returned / keep fine
  RETURNED --> [*]
```

EXPECTED OUTPUT




```
README.md · Review report (excerpt)
1. Two initial nodes           -> "one initial node":
                                  split into two activities or add a merge
2. 23 actions                  -> "~15 actions":
                                  extract "Escalate fine" as a call behaviour
action
3. [ok] / [maybe], no [else]   -> "guards complete and exclusive":
                                  [d < 3], [3 ≤ d < 7], [7 ≤ d < 30], [d ≥ 30]
4. Fork: 4 branches, 2 joined  -> "every fork has its join":
                                  join all four, or end branches with ⊗
```

```
5. 14 "returned" edges          -> one interruptible region + accept event
6. Lane "Misc"                  -> lanes are parties: System, Notification,
Member
7. No final node                -> add ⊙ after "Close overdue case"
8. 9 edge crossings             -> main flow top-down, escalation left to right
```

Show hints

- Timed escalation is one region: three hourglass nodes in sequence, each followed by its action; the interrupting edge from *Book returned* leaves the region wherever the token currently is.
- Idempotence: store `reminderSentOn`, `fineStartedOn`, `suspendedOn` on the loan and guard each action with "not yet done".
- In the state diagram the boxes are states of one `Loan` and the edges are events; in the activity diagram the boxes are work and the edges are "then". Say which question each answers.
- Scheduling needs `@EnableScheduling` on a configuration class; a Spring Boot 4 project with `spring-boot-starter-webmvc` is enough for the sketch, no database required.

ACCEPTANCE CHECKLIST

- [ ] The overdue diagram uses three accept time events inside one interruptible region with a "Book returned" accept event and a lightning edge.
- [ ] `OverdueJob` compiles, uses `@Scheduled`, has complete and exclusive day guards and is idempotent.
- [ ] The `stateDiagram-v2` renders and the README comparison names at least three concrete differences.
- [ ] The review report lists at least eight problems with the rule violated, and the redrawn diagram passes the readability checklist.

Grading rubric

Component	Points	Full marks when…
Task 1 · Borrow a book	10	Correct notation, one decision with proper guards and merge, `borrow()` mirrors the actions
Task 2 · Return with fines	15	Three-way decision with complete and exclusive guards, guards as boolean methods, boundary tests and the "exactly one guard" test pass

Component	Points	Full marks when…
Task 3 · Reservation fulfilment	20	Four lanes, fork/join, loop via merge; both concurrent implementations pass the ordering and failure tests
Task 4 · Bulk import	20	Object nodes, expansion region, interruptible region and exception handler drawn correctly and implemented with per-row `catch` and cancellation
Task 5 · Membership renewal	25	End-to-end process, code structure mirrors the diagram, one passing test per path
Code / diagram quality	10	Verb-noun names, ≤ 15 actions per diagram, consistent ids and lane names, README complete, PNGs match the .mmd sources
★ Challenge (extra credit)	+20	Timed overdue diagram, idempotent `@Scheduled` sketch, state-vs-activity comparison, review report with redraw
Total	**100 (+20)**	

Automatic deductions: missing initial or final node (−3 per diagram) · guards that overlap or leave a gap (−5 per decision) · fork without matching join (−5) · alternative paths entering an action directly instead of through a merge (−3 each) · actions not named verb + noun (−1 each, max −6) · a diagram submitted only as PNG without its source text (−5) · Java whose step order differs from the diagram (−5).

Submission

1. Repository layout:

```
library-activities/
├── README.md                          # guard tables, comparison, review
report
├── pom.xml
├── diagrams/
│   ├── task1-borrow.mmd / .png
│   ├── task2-return.mmd / .png
│   ├── task3-reservation.mmd / .png
│   ├── task4-import.mmd / .png
│   ├── task5-renewal.mmd / .png
│   ├── challenge-overdue.mmd / .png       (optional)
│   ├── challenge-loan-states.mmd / .png   (optional)
```

```
    │   └── challenge-review.mmd / .png        (optional)
    └── src/
        ├── main/java/edu/itc/library/...
        └── test/java/edu/itc/library/...
```

2. Verification from a clean checkout: `mvn -q test` must pass, and `for f in diagrams/*.mmd; do npx -p @mermaid-js/mermaid-cli mmdc -i "$f" -o "${f%.mmd}.png"; done` must regenerate every PNG without errors.
3. Branch `lab-09`, merge request titled `Lab-09 – <your name>`; paste the Task 5 PNG into the merge request description.

Back to Chapter 09 slides | Retake the chapter quiz | ← Lab-08